

Documentación Trabajo RL

1.- Introducción

1.1.- Objetivo del proyecto

1.2.- Tecnologías usadas

2.- Conceptos Técnicos

2.1.- Conceptos de Reinforcement Learning (Aprendizaje por Refuerzo)

2.2.- Entorno

2.2.1.- Circuitos

2.2.2.- Espacio de Observación

2.2.3.- Espacio de Acciones

2.3.- Algoritmos

2.3.1.- A2C

2.3.2.- SAC

2.4.- Métricas Más Importantes

2.4.1.- Reward (Recompensa)

2.4.2.- Coeficiente de Entropía

2.4.3.- Pérdida del Valor (Value Loss)

3.- Desarrollo del Proyecto

3.1.- Usando Imágenes y CNN

3.1.1.- El óvalo

3.1.2.- Circuitos procedimentales

3.2.- Usando Vectores y MLP

3.2.1.- Primer entrenamiento

3.3.- Pedagogía de la IA

3.3.1.- Reward Shaping (Distribución de Recompensa)

3.3.2.- Curriculum Learning

4.- Conclusiones y futuro

1.- Introducción

El presente documento detalla el desarrollo de un **Agente Robótico Autónomo** entrenado mediante **Aprendizaje por Refuerzo (Reinforcement Learning)**. El proyecto se sitúa en un entorno de simulación donde un vehículo debe aprender a navegar por circuitos complejos, optimizando su trayectoria y velocidad de manera totalmente autodidacta.

Este trabajo representa la culminación del Ciclo Formativo de Grado Superior en Big Data e Inteligencia Artificial, integrando conocimientos de sistemas de aprendizaje automático, programación avanzada en Python y despliegue de modelos.

1.1.- Objetivo del proyecto

El objetivo principal es diseñar, entrenar y evaluar un modelo de **Deep Reinforcement Learning** capaz de resolver un problema de navegación continua. Los objetivos específicos se desglosan en:

- **Desarrollo de un entorno personalizado:** Crear una simulación compatible con el estándar **Gymnasium** que presente retos de dificultad progresiva (curriculum learning).
- **Comparativa de Arquitecturas:** Evaluar el rendimiento entre redes neuronales convolucionales (**CNN**) para procesamiento de imágenes frente a redes densas (**MLP**) para vectores de estado.
- **Optimización del Agente:** Implementar y ajustar algoritmos avanzados como **SAC** (Soft Actor-Critic) y **A2C** para lograr una conducción fluida y eficiente en circuitos no vistos anteriormente (generalización).
- **Pipeline MLOps:** Versionar el modelo y los experimentos para asegurar la reproducibilidad de los resultados.

1.2.- Tecnologías usadas

Para el desarrollo del proyecto se ha seleccionado un stack tecnológico de nivel industrial, adaptado para su ejecución en entornos de computación locales (ordenadores personales). A diferencia de la propuesta inicial que contemplaba el uso de infraestructura del centro o la nube, hemos optado por optimizar el desarrollo para hardware propio, lo que ha condicionado y motivado gran parte de nuestras decisiones de optimización.

- **Hardware de Desarrollo:** El entrenamiento y las pruebas se han realizado íntegramente en **ordenadores personales**. Esta limitación de recursos fue el motor principal para pivotar desde el uso de imágenes (computacionalmente costosas) hacia un espacio de observación vectorial, buscando una mayor eficiencia en los tiempos de iteración.
- **Lenguaje de Programación: Python**, como eje central de todo el desarrollo.
- **Frameworks de IA:**
 - **Keras / TensorFlow:** Para la construcción de las arquitecturas de redes neuronales (DQN, CNN y MLP).
 - **Stable-Baselines3:** Para la implementación de los algoritmos de RL y wrappers de procesamiento.
- **Simulación: Gymnasium (OpenAI Gym) y Pygame** para la creación y renderizado de los circuitos y las físicas del coche.
- **Visualización y Despliegue:**
 - **Streamlit:** Para la creación de un dashboard interactivo que muestra las métricas de entrenamiento y vídeos de los episodios.
 - **FastAPI:** Para servir el modelo entrenado a través de una interfaz de programación de aplicaciones.
 - **Docker:** Para la contenedorización de la solución, garantizando que el entorno de ejecución sea idéntico en cualquier máquina.
- **Análisis y Monitorización:**
 - **TensorBoard / MLflow:** Para el seguimiento de las curvas de recompensa y entropía.

- **R (ggplot2):** Utilizado para el análisis estadístico avanzado del rendimiento del agente y la generación de gráficos profesionales para esta memoria.

2.- Conceptos Técnicos

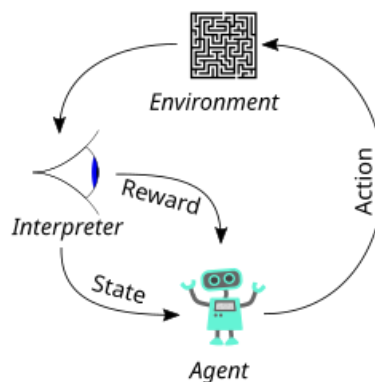
En este punto se contemplan los conceptos técnicos más relevantes del Reinforcement Learning (a partir de ahora RL) para poder comprender este documento.

2.1.- Conceptos de Reinforcement Learning (Aprendizaje por Refuerzo)

El RL es una técnica de Machine Learning (Aprendizaje Automático) cuyo objetivo es que un agente pueda aprender a desarrollar tareas de forma autónoma al ser entrenado en un entorno o simulación similares a los de su área de trabajo posterior.

La idea detrás de esto es que el agente observe el estado de su entorno, pruebe a realizar acciones y reciba una retroalimentación sea en forma de recompensa (si son la acción correcta) o penalización (si son la acción errónea), lo que se traduce en una puntuación numérica. El objetivo del agente siempre será maximizar esta recompensa, aunque esto puede llevar a resultados no esperados si el entorno o la recompensa no están bien ajustados:

- El agente queda "asustado" (Óptimo Local): En este caso el agente prefiere no realizar acciones y no perder puntos a arriesgarse a tener una penalización muy severa.
- El agente encuentra un vacío un legal (Reward Hacking). Si el entorno y las recompensas no están bien hechos, el agente puede encontrar formas de maximizar su recompensa sin cumplir con los objetivos esperados (ej: se queda parado, avanza y retrocede, da vueltas, etc.).



Estos conceptos serán tratados en profundidad más adelante.

2.2.- Entorno

El entorno es el mundo dónde se mueve el agente, con sus físicas, normas y recompensas tras realizar acciones. En el caso del proyecto, este entorno está compuesto por los circuitos de carreras.

Es importante destacar que las físicas del entorno de este proyecto son muy simples y no buscan ser un reflejo fiel de la realidad. Por ejemplo, en el entorno existe una fricción no se calcula dinámicamente usando el peso del coche o el agarre de sus neumáticos; en el entorno es un valor fijo que decelera progresivamente cuando el coche no está acelerando.

Tampoco existe una ley de la gravedad y, por lo tanto, no hay transferencia de pesos del coche, por lo que no es del todo esperable que el agente trace las curvas, de exterior → vértice → exterior, como se trazarían en la vida real. - *Realmente este tipo de trazadas también responden a un problema geométrico, pero el punto es que las físicas son ultra simplificadas en comparación con las reales*-.

Además, los entornos pueden ser:

- **Deterministas:** Si realizar una acción da el mismo resultado siempre. Ej: Acelerar un 20% aumenta la velocidad un 20% siempre, independientemente de todo. Es el caso de este proyecto.
- **Estocásticos:** Si realizar una acción no garantiza siempre el mismo resultado. Ej: Accionar "Subir" puede acabar con el agente en la celda de la izquierda. No es el caso de este proyecto, pero es un fenómeno interesante que se explica perfectamente en el entorno Frozen Lake de Gymnasium. Como nota, este tipo de entornos enseñan a los agentes a reaccionar ante adversidades de la vida real como una ráfaga de viento repentina para un dron.

Aunque existen diferentes clase de "Entornos" en el código fuente del proyecto, sólo el de la clase `entorno_def.py` es el definitivo y usado en este proyecto. El resto pueden ser considerarse tests previos para llegar a este.

2.2.1.- Circuitos

Simulando un coche de carreras, el agente se mueve por circuitos como si de un piloto se tratase. Para el entrenamiento se han diseñado dos tipos de circuitos:

- **Óvalo:** Un óvalo perfecto y estático, creado principalmente como entorno de pruebas inicial para que el agente dé sus primeros pasos, entienda las físicas y acciones que puede realizar y las recompensas asociadas a estas antes de enfrentarse a un reto real. A esta técnica se le llama Aprendizaje Curricular (Curriculum Learning) y será tratada más adelante.
- **Circuitos procedimentales:** Estas pistas se generan aleatoriamente mediante un algoritmo, por lo que nunca existirán dos circuitos idénticos. Tienen diferentes tipos de curvas y suponen un reto a la generalización del modelo. Si el agente es capaz de navegar por estos circuitos quiere decir que su generalización es buena y que no sólo ha aprendido a correr en el óvalo (sobreajuste, *overfitting*), sino que ha aprendido a conducir de verdad.

Estas pistas son generadas en la clase `track_gen.py`.

Los circuitos cuentan con una serie de puntos de control (Checkpoints) que el algoritmo va a buscar para poder calcular su próximo objetivo.

Ambos tipos de circuitos se han diseñado bajo la misma idea de *generación mediante coordenadas polares*, que se resume en:

1. **Distribución base:** Se genera un número de puntos y se distribuyen de forma equitativa en un espacio circular. En el caso de los circuitos procedimentales, se aplica un ruido al radio de cada punto para evitar crear círculos perfectos y añadir aleatoriedad en las curvas.
2. **Suavizado:** Se suaviza la trayectoria creada entre estos puntos. De no hacerlo, la superficie sería un polígono irregular con unas curvas acabadas en picos. Para esto se aplica una *spline* o curva matemática para crear puntos entre los puntos ya existentes, generando así 100 puntos por pista que servirán de Checkpoints para el agente durante el entrenamiento.
3. **Extrusión:**
 - a. Se calcula la tangente del punto para deducir hacia donde avanza el circuito.
 - b. Se normaliza (longitud = 1) para tener valores estándar y evitar errores por las diferentes distancias.
 - c. Se calcula la normal (se gira el vector obtenido 90°) y se obtiene un vector perpendicular que apunta directamente a ambos lados de la pista.
 - d. Se aplica la mitad de un ancho obtenido por parámetro como distancia de este punto al nuevo punto, que será el punto interior/exterior del circuito y se dibuja la línea. Este proceso se realiza por cada punto del circuito.

Nota: El proceso del óvalo es similar, pero en lugar de 100 Checkpoints se generan 50 y no se aplica ningún ruido.

Como apunte: Existió un problema prolongado en el que los circuitos procedimentales no se generaban correctamente. Es por ello que existe una salvaguarda en forma de función `generate_valid_track()` que comprueba que el circuito sea válido mediante el cálculo de los ángulos entre sus Checkpoints (si es demasiado grande, el circuito no será transitable). De no serlo devuelve un óvalo para esta iteración. En este punto del proyecto, casi el 100% de los circuitos generados procedimentalmente son válidos.

2.2.2.- Espacio de Observación

El Espacio de Observación se refiere a todo aquello que el agente percibe del entorno para poder realizar sus acciones. Haciendo la analogía, puede definirse como los sentidos del modelo.

Aunque en los puntos 3.1.- Usando Imágenes y CNN y 3.2.- Usando Vectores y MLP se hablará más en detalle de los espacios de observación utilizados podemos resumirlos en este punto:

- **Imágenes:** El algoritmo “ve” el entorno a través de capturas procesadas por una Red Neuronal Convolutiva (CNN). Se realiza una captura, se transforma en algo más ligero y fácil de procesar para la red (se aplica una escala de grises y se reescala) y el agente toma una decisión analizando esos píxeles. Este tipo de espacio de observación es mucho más pesado computacionalmente que los vectores convencionales debido a la

cantidad de píxeles que componen una imagen, especialmente cuando el modelo debe procesar decenas por segundo.

- **Vectores:** El algoritmo lee directamente variables numéricas puras que reflejan su propio estado y el del entorno. En el caso del proyecto se ha construido una observación ligera y simple compuesto por: posición, orientación y velocidad del coche, y las coordenadas de su próximo Checkpoint.

2.2.3- Espacio de Acciones

El Espacio de Acciones compone las diferentes acciones y controles que puede realizar el agente sobre el entorno. Siguiendo con las analogías, puede definirse como "el mando" del modelo si el entorno fuese su juego.

Existen dos espacios de acciones:

- **Discretos:** El agente debe elegir una acción concreta dentro de un conjunto finito de opciones mutuamente excluyentes. Esto quiere decir, que nunca pueden realizarse simultáneamente y siempre se realizan de forma secuencial. Estas acciones suelen estar representadas por números enteros que suelen actuar como valores booleanos o digitales (0/1, apagado/encendido). Ej: Un agente que navega por un laberinto y en cada paso sólo puede elegir entre moverse arriba, abajo, a la izquierda o la derecha.
- **Continuos:** El agente puede realizar acciones simultáneas y con diferentes grados de intensidad. Estas acciones suelen estar representadas como números decimales. Ej: El coche puede acelerar al 80% y girar un 20% al mismo tiempo.

En el caso del proyecto, el espacio de acción es continuo. El agente tiene a su disposición un vector de acción de dos dimensiones (dos valores numéricos) que van desde **-1.0** hasta **1.0**:

- **Girar (Componente 1):** Define la dirección del volante. Un valor negativo indica un giro a la izquierda, mientras que un valor positivo indica un giro a la derecha. La intensidad del giro es proporcional al valor absoluto del número. Ej: **-0.30** → girar a la izquierda un 30%.
- **Acelerar/Frenar (Componente 2):** Controla los pedales del coche. Si el valor es mayor a **0.0**, el agente acelera y añade velocidad. Si el valor es negativo, el agente frena y decelera. Similar al giro, la intensidad de aceleración y frenado es proporcional al valor absoluto del número.

Nota: Inicialmente el agente podía dar marcha atrás al dar un valor negativo (lo que puede considerar "Acelerar" de forma negativa). Sin embargo, lo usaba para maximizar su recompensa al ser penalizado por no moverse (se hablará de ello más en profundidad en el punto 3.3.1.- Reward Shaping).

2.3.- Algoritmos

En el desarrollo de este proyecto, se han seleccionado algoritmos capaces de gestionar **espacios de acción continuos** (giro y aceleración gradual), lo que permite una conducción mucho más fluida que los sistemas discretos.

2.3.1.- A2C

El algoritmo A2C es la versión síncrona del paradigma Actor-Crítico. Su arquitectura se divide en dos redes neuronales que trabajan en simbiosis para optimizar el aprendizaje:

- **El Actor (La Política):** Se encarga de mapear el estado actual (los vectores de posición y velocidad) hacia una acción específica. Al trabajar en un espacio continuo, el actor no elige una "tecla", sino que define una distribución de probabilidad (normalmente una Gaussiana) de la que se extraen los valores de giro y aceleración.
- **El Crítico (La Función de Valor):** Su función no es decidir acciones, sino evaluar lo bueno que es el estado en el que se encuentra el agente. Estima la recompensa total esperada a futuro.
- **La Ventaja (Advantage):** A2C mejora el aprendizaje mediante el cálculo del *Advantage* ($A = Q(s,a) - V(s)$). Esto le indica al actor si la acción tomada fue mejor que el promedio de acciones posibles en ese estado. Si el resultado es positivo, el actor refuerza esa estrategia; si es negativo, la descarta.
- **Uso en el proyecto:** Se ha utilizado como un *baseline* de estabilidad. Al ser síncrono, espera a que el agente complete sus interacciones con el entorno para actualizar los pesos de la red, lo que garantiza gradientes más consistentes, aunque a veces más lentos que métodos asíncronos.

2.3.2.- SAC

SAC es uno de los algoritmos más avanzados para RL en entornos continuos y ha sido el motor principal de nuestras pruebas más exitosas. Su característica diferencial es la **Optimización de la Entropía Máxima**.

- **Objetivo Dual:** Mientras que los algoritmos tradicionales solo buscan maximizar la **recompensa acumulada**, SAC busca maximizar la **recompensa + la entropía**. Esto incentiva al coche a explorar de forma activa, evitando que se quede estancado en una estrategia mediocre pero "segura" (como quedarse quieto para no perder puntos).
- **Arquitectura Off-Policy:** SAC utiliza un *Replay Buffer* (memoria de experiencias). Esto permite que el algoritmo aprenda de datos pasados, lo que mejora drásticamente la eficiencia computacional en comparación con A2C.
- **Aprendizaje Automatizado del "Temperatura":** SAC ajusta automáticamente cuánto peso darle a la exploración (entropía) frente a la explotación (recompensa). Esto es vital en nuestros circuitos procedimentales, donde el agente necesita mucha flexibilidad para adaptarse a curvas nunca antes vistas.

2.4.- Métricas Más Importantes

Para evaluar si el entrenamiento progresa adecuadamente, monitorizamos variables clave a través de **TensorBoard** y el dashboard de **Streamlit**.

2.4.1.- Reward (Recompensa)

Es el indicador principal de rendimiento. Representa la suma total de recompensas obtenidas por el agente en un episodio completo.

- **Curva de Aprendizaje:** Una curva saludable debe mostrar un crecimiento logarítmico. Al principio, las oscilaciones son bruscas debido a la exploración aleatoria. Un éxito en el proyecto se identifica cuando la recompensa media por episodio se estabiliza en valores positivos altos, indicando que el coche ha aprendido a completar vueltas sin salirse de la pista.
- **Relación con el Reward Shaping:** Como hemos visto en la "Sexta Iteración", el éxito de esta métrica depende de que la recompensa por pasar *checkpoints* sea superior a la penalización por inactividad. Si el *reward* acumulado no sube a pesar de los pasos de entrenamiento, nos indica un fallo en el diseño del entorno o un agente "asustado".

2.4.2.- Coeficiente de Entropía

La entropía es una medida de la aleatoriedad de las acciones del agente. Es la métrica que nos indica qué tan "segura" está la IA de sus decisiones.

- **Entropía Alta (Fase de Exploración):** Al inicio del entrenamiento, la entropía debe ser alta. El agente no sabe conducir, por lo que prueba todas las combinaciones de giro y aceleración posibles. Es una fase de "caos necesario".
- **Entropía Baja (Fase de Explotación):** A medida que el entrenamiento avanza (especialmente en el paso de los 50k con vectores), la entropía cae gradualmente. Esto significa que el agente ha identificado qué acciones maximizan su recompensa y empieza a repetirlas con seguridad.
- **Interpretación en el proyecto:** Si la entropía cae a cero demasiado rápido (colapso de la política), el coche se vuelve rígido y no sabe reaccionar ante curvas nuevas. Si se mantiene siempre alta, el coche nunca llega a "aprender" y conduce de forma errática para siempre. El equilibrio buscado es una caída suave que coincida con la subida de la recompensa.

2.4.3.- Pérdida del Valor (Value Loss)

Esta métrica, específica de arquitecturas como SAC y A2C, mide qué tan bien está el "Crítico" prediciendo las recompensas futuras.

- **Importancia:** Si el *Value Loss* es muy alto, significa que el crítico no entiende el entorno y, por tanto, le está dando consejos erróneos al actor. Una reducción en esta pérdida es una señal de que el modelo está comprendiendo las "físicas" y reglas del circuito que hemos diseñado.

3.- Desarrollo del Proyecto

En este punto se comenta cómo ha sido el desarrollo del proyecto, qué principales problemas han surgido y cuáles han sido las soluciones aportadas. Es un punto un tanto más narrativo, personal e informal, por lo que se ruega comprensión si ciertas partes contienen alusiones al "nosotros" y las explicaciones se basan más en la experiencia que en la teoría técnica.

3.1.- Usando Imágenes y CNN

Inicialmente, la idea del proyecto era utilizar un espacio de observación compuesto por imágenes. Estas imágenes eran capturas directas del entorno de simulación, que era procesadas mediante una CNN y hacían que el algoritmo “viese”.

Sin embargo, las imágenes tienen un problema inherente: son excesivamente pesadas computacionalmente. Para entender esto, primero tenemos que desgranar *¿Qué es una imagen?*

Si invadir el campo de la fotografía y el audiovisual, matemáticamente una imagen se define como una matriz de píxeles compuesta por varios canales (3, *RGB* - Rojo, Verde, Azul). En el caso del proyecto, las ventanas de los circuitos son de 800×800 píxeles.

$$800_{\text{píxeles}} \times 800_{\text{píxeles}} = 640.000_{\text{píxeles totales}} \times 3_{\text{canales}} = 1.920.000_{\text{entradas}}$$

Cada uno de estos casi dos millones de valores representan una entrada o *input* individual que la red debe procesar en cada iteración. Es fácil ver que la potencia de cálculo requerida para un problema así es brutal. Es por ello que es necesario usar una Red Neuronal Convolutiva (CNN), creadas específicamente para extraer esas características espaciales y reducir la dimensionalidad de este tipo de problemas.

Aún así, la CNN necesita una simplificación de las imágenes para procesar rápidamente cada imagen, por lo que decidimos aplicar un preprocesamiento. Para aligerar el proceso, se reducía la resolución de la captura a una imagen de 64×64 píxeles con un sólo canal. Es decir, las imágenes eran muchísimo más pequeñas y en escala de grises. Estas imágenes son computacionalmente más ligeras y fáciles de procesar para la CNN que una completa y eran capaces de representar el coche, el circuito y sus límites.

$$64_{\text{píxeles}} \times 64_{\text{píxeles}} = 4.096_{\text{píxeles totales}}$$

Una vez optimizadas las imágenes, surgió un nuevo problema: el agente no tenía concepto de la velocidad ni del giro. Al ver tan sólo un fotograma de la simulación, el agente no era capaz de entender su velocidad actual ni su grado de giro.

La solución adoptada fue aplicar una técnica conocida como *Frame Stacking* (Apilamiento de Fotogramas). La idea es modificar el entorno para que en lugar de enviar una sola imagen, envíe 4 de golpe en un sólo paquete para que las procese la red neuronal en conjunto. Esto consiguió que el agente frenase ante posibles salidas y mejorase en relación al punto de partida, permitiéndole comprender su velocidad y la dirección de sus movimientos. Pero como consecuencia la coste computacional subió y la velocidad de entrenamiento volvió a bajar.

$$4_{\text{fotogramas}} \times 4.096_{\text{píxeles}} = 16.384_{\text{entradas}}$$

Esta implementación se logró utilizando los *wrappers* de *stable_baselines3*, que facilitan en gran medida este proceso. Este “envoltorio” se debía realizar tanto en el entorno de entrenamiento como el de monitorización/visualización.

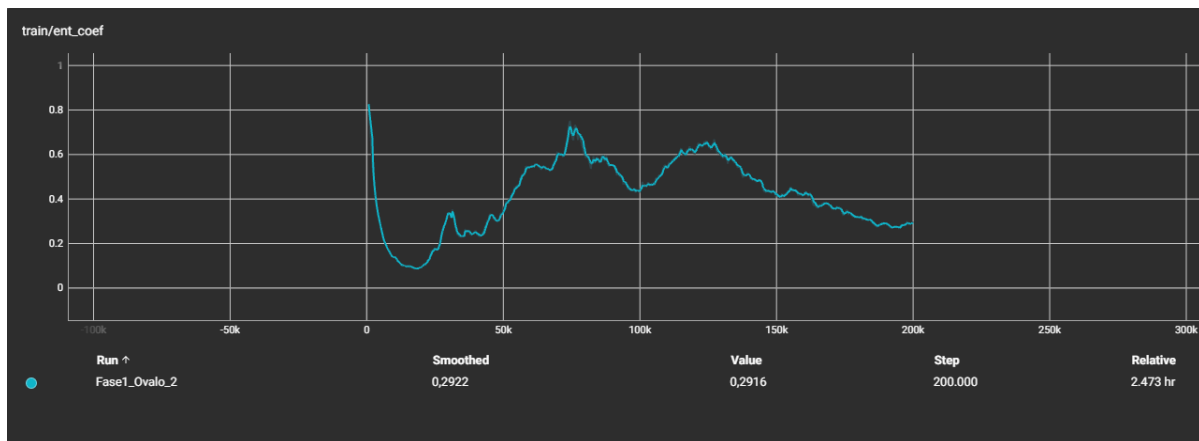
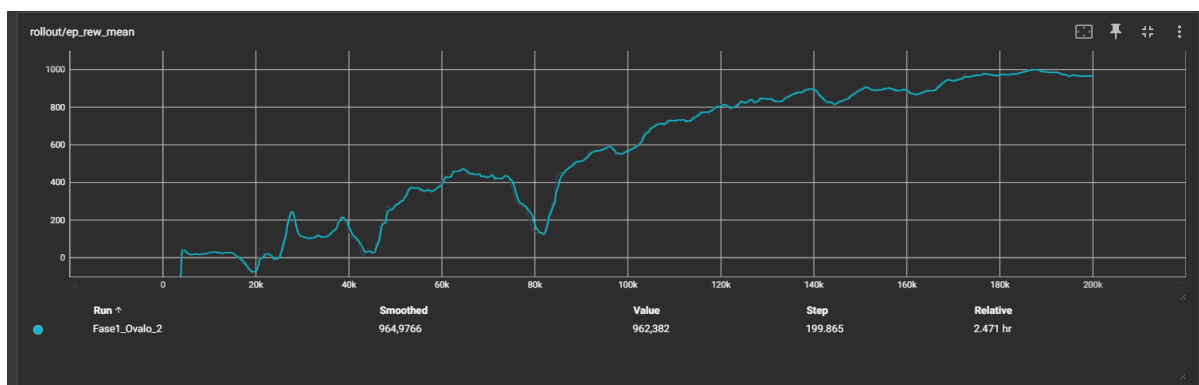
Como apunte, este problema es bastante común en el RL con imágenes (nosotros no lo sabíamos hasta entonces). Este paper de Simeet Nayan ilustra bastante bien en qué consiste el problema y aplica la solución que se considera estándar en la industria.

Nota: Las gráficas mostradas en este punto fueron realizadas con recompensas que variaban entre -100 y 100 y que fueron cambiadas posteriormente. Para consultar los cambios en la recompensa se recomienda ver el punto [3.3.1.- Reward Shaping](#). Todo este punto ha sido realizando utilizando el algoritmo SAC.

Los resultados fueron prometedores, aunque lentos. Por ello decidimos hacer un entrenamiento en el óvalo, que proporcionaba una imagen prácticamente estática y un entorno muy fácil de navegar.

3.1.1.- El óvalo

El primer entrenamiento constó de 200.00 pasos (representados como 200k) y duró aproximadamente dos horas y media:

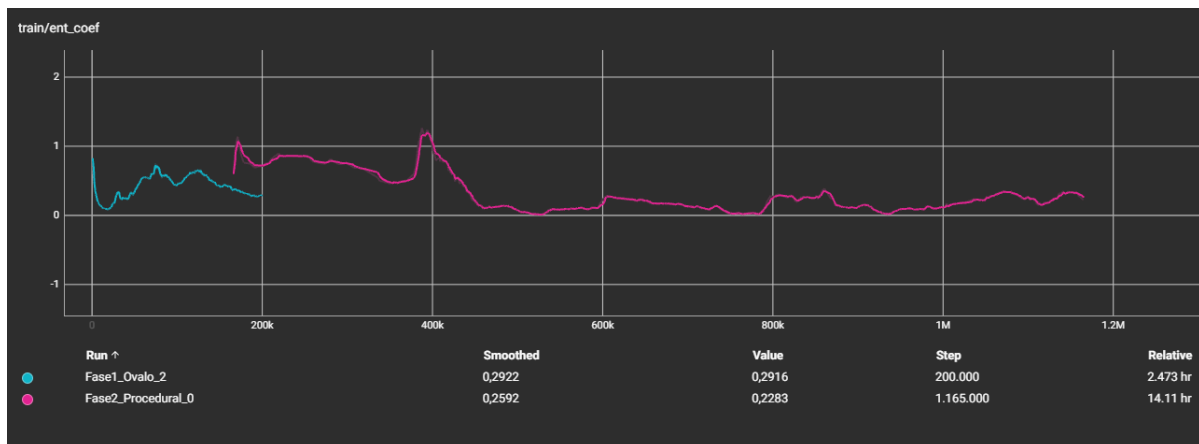
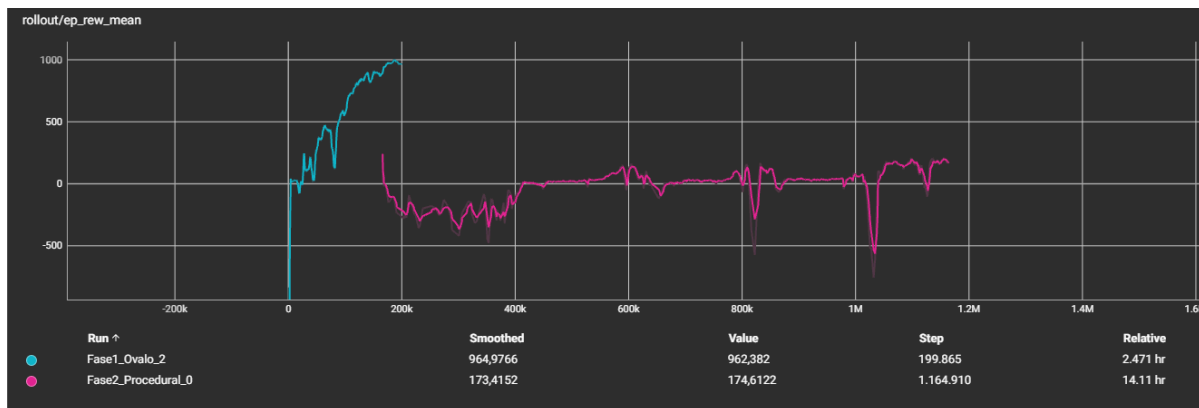


- **Reward (Recompensa):** El agente consiguió completar el circuito de forma consistente alrededor del paso 120k. A partir de ahí, la curva se estabiliza y sube lentamente, indicando que el agente ya sabía sobrevivir y se dedicaba a optimizar su trazada y, por tanto, su recompensa (cuantos menos acciones hiciera, más puntos ganaba).
- **Entropía:** La gráfica de la entropía empieza a bajar de forma progresiva, lo que refleja que el agente pasó de una fase de exploración (acciones aleatorias) a una de explotación (confiar en su propia experiencia) y ejecutaba las acciones que le habían dado resultado anteriormente.

Al visualizarlo, efectivamente, el agente podía completar el óvalos sin problemas. Esto fue un “chute” de optimismo que nos motivó al siguiente paso: usar este modelo preentrenado para realizar un entrenamiento largo en las pistas procedimentales (técnica del Curriculum Learning).

3.1.2.- Circuitos procedimentales

Este segundo entrenamiento constó de 1 millón de pasos y tardó algo más de 14 horas. Al arrancar desde el mejor modelo obtenido en el óvalo (que no era el que había dado el último paso como tal), las gráficas se solapan un poco.



- Al principio la recompensa cae en picado y la entropía sufre un pico al alza drástico. Esto es lo que se conoce como *Distribution Shift* (Cambio de Distribución). El modelo se había sobreajustado al óvalo, con únicamente curvas de derechas, y fue incapaz de procesar las sinuosas curvas de las curvas de los nuevos circuitos. Dicho de otra manera, entró en pánico y empezó a realizar acciones aleatorias.

Este fenómeno también puede deberse al sobreajuste de la CNN para leer las imágenes, ya que los óvalos no variaban nunca y la imagen era siempre la misma (a excepción del coche que se movía por ellas).

- La entropía se estabilizó alrededor del paso 500k y el agente recuperó algo de recompensa.

Al visualizar este algoritmo el resultado fue decepcionante después de tanto tiempo. El agente era incapaz de hacer una sola curva. Tuvo tiempo suficiente y pasos suficientes de poder adaptar su política, pero no lo hizo. Cayó en un óptimo local (el agente asustado del punto 2.1), consiguiendo la recompensa por pasar unos pocos checkpoints fáciles y luego a quedarse quieto para no perderla. Fue un fracaso absoluto.

Después de varios intentos de cambios tanto en el entorno como la observación y la recompensa, fuimos incapaces de hacer generalizar un algoritmo entrenado exclusivamente con imágenes. El resultado era siempre el mismo. Sólo con esto perdimos fácilmente una semana y media entre entrenamientos y cambios.

Es importante decir que al entrenar un algoritmo el ordenador dónde se estuviera utilizando quedaba completamente inutilizado porque la potencia cálculo requerida impedía realizar ninguna otra cosa y, como es de suponer, tenemos otras asignaturas y tareas que realizar.

Así que cambiamos el enfoque.

3.2.- Usando Vectores y MLP

Tras el fracaso que supuso intentar entrenar un modelo exclusivamente con imágenes y el valioso tiempo perdido en el proceso, tuvimos miedo de no llegar a entrenar nada más que un puñado de expectativas. Era experiencia valiosa, sí, pero al final del día eran resultado decepcionantes.

Cambiamos radicalmente el enfoque y pasamos a un espacio de observación compuesto por un vector de 7 dimensiones:

- Posición en X.
- Posición en Y.
- Velocidad actual.
- Orientación en X.
- Orientación en Y.
- Coordenada del próximo Checkpoint en X.
- Coordenada del próximo Checkpoint en Y.

Este cambio supuso un avance más monumental:

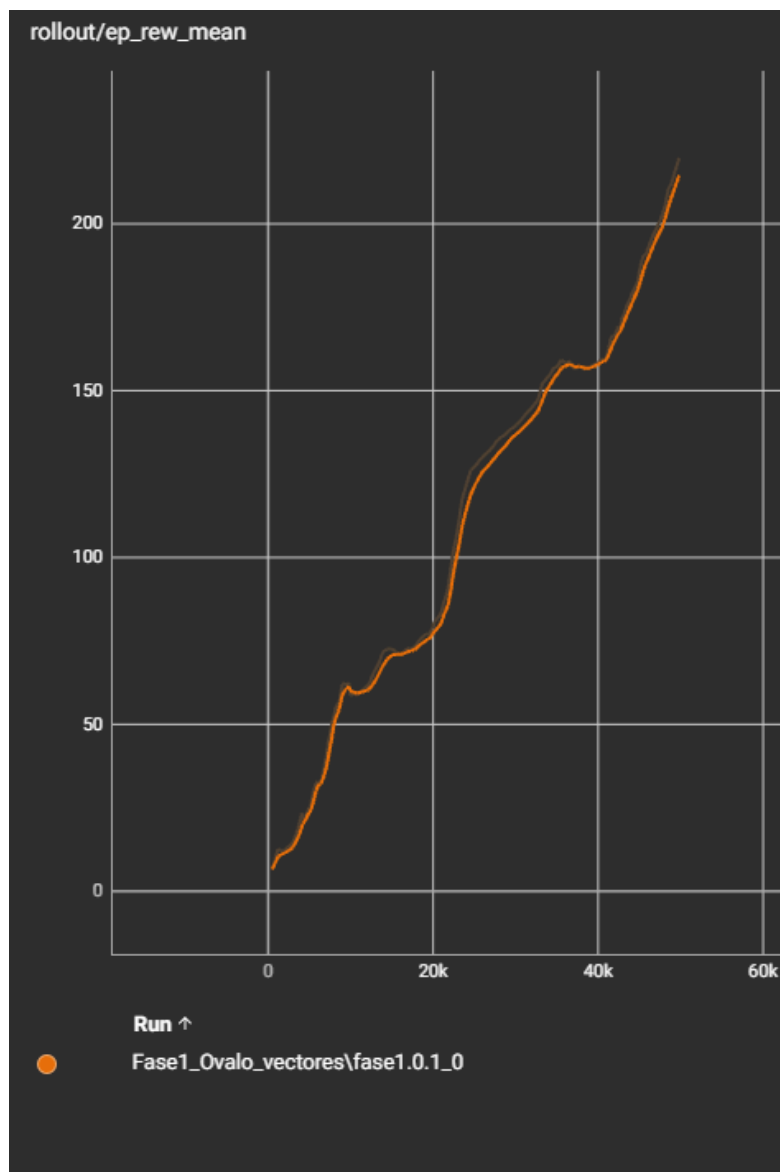
- **Eficiencia computacional:** El espacio de observación pasó de 16.384 entradas individuales producidas por las imágenes y el frame stacking a tan solo 7.
- **Cambio de arquitectura:** Al pasar a vectores de números simples pudimos sustituir la compleja CNN por una MLP (Red Densa Profunda) convencional. Estas redes neuronales son muchísimo más rápidas, ligeras y fáciles de entrenar, lo que se tradujo en más eficiencia computacional.
- **Generalización:** Aunque no es el objetivo del proyecto, el uso de vectores mejora exponencialmente la capacidad de generalización teórica del modelo. Si intentásemos usar el algoritmo entrenado con las imágenes en un coche real, fallaría estrepitosamente, ya que no sabría interpretar las texturas o las luces del mundo física porque fue

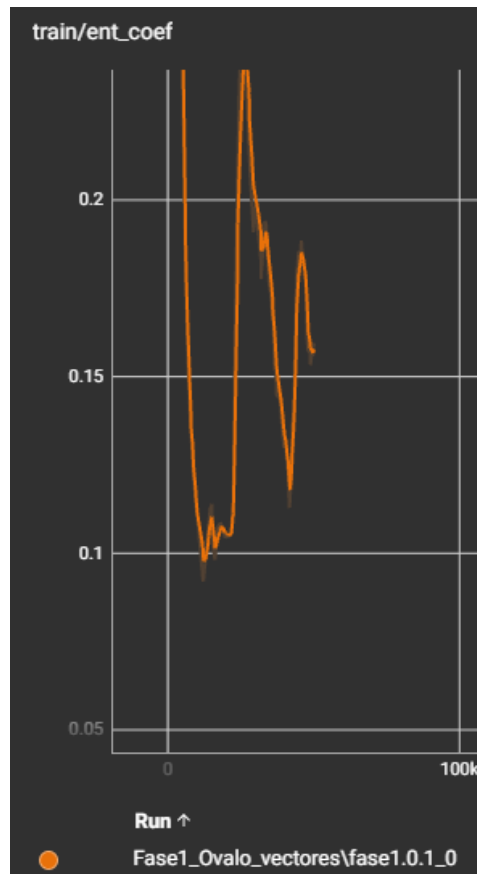
entrenado con imágenes de polígonos rellenos con colores sólidos y lisos. Sin embargo, las coordenadas de un coche, su velocidad o su orientación si son parámetros que se presentan en la realidad, por lo que el conocimiento del modelo puede llegar a ser transferido a la realidad.

El primer entrenamiento dio los resultados más prometedores hasta el momento.

3.2.1.- Primer entrenamiento

Este entrenamiento constó de 50k pasos para comprobar que todos los cambios funcionaban correctamente. La primera pequeña victoria fue en el rendimiento: el entrenamiento duró apenas 25 minutos.





- **Recompensa:** Subió rápidamente. En apenas 10k pasos el agente era capaz de entender como se movía en el mundo, que le daba recompensas y qué lo penalizaba. Alrededor del paso 40k era capaz de completar el circuito y empezó su fase de explotación.
- **Entropía:** Empezó en un valor altísimo (`0.8`), indicando máxima exploración, pero bajó súbitamente en los primeros pasos. Esto indicaba que el agente ganaba confianza en su conocimiento rápidamente y no requería de probar acciones nuevas.

Una grandísima noticia si tenemos en cuenta el cambio gigante que había sufrido el entorno y el nuevo espacio de acciones. Aún con todo, estas gráficas ocultaban un éxito aún mayor:

- Habíamos aprovechado el cambio de observación y arquitectura para cambiar también la recompensa obtenida. En un primer instante esta iba de `-100` a `100` , pero el estándar con estos algoritmos es que vaya en incrementos menores para no desestabilizar los gradientes de la red neuronal. La nueva recompensa era de `-10.0` a `10.0` , con micro-recompensas en forma de décimas entre medias.
- Aunque el nombre representado sea *Fase1_Ovalo...* lo cierto es que, por un despiste de configuración, lo estábamos entrenando directamente en las pistas procedimentales.

Es decir, el agente estaba aprendiendo excepcionalmente rápido, dando resultados increíblemente buenos, en el entorno más difícil sin necesidad de pasar por la pista de pruebas. El cambio de imágenes a vectores había dotado al modelo de una capacidad de generalización inmediata.

3.3.- Pedagogía de la IA

En este punto se trata como la Inteligencia Artificial aprende de sus acciones mediante el Reward Shaping (Distribución de la Recompensa) y la técnica del Curriculum Learning (Aprendizaje Curricular).

3.3.1.- Reward Shaping (Distribución de Recompensa)

El Reward Shaping hace referencia a cómo se diseña y se distribuye la recompensa en función de las acciones del agente. Cabe recordar que los algoritmos de RL aprenden mediante prueba y error, intentando maximizar una recompensa que se obtiene realizando acciones.

A lo largo del desarrollo de este proyecto la distribución de la recompensa ha cambiado múltiples veces. En este punto se detallan exclusivamente los cambios más relevantes y cómo afectaron a los algoritmos (se han omitido ajustes menores que no supusieron variaciones importantes).

Nota: Toda la distribución de la recompensa ha sido ajustada y probada haciendo las pruebas con el algoritmo SAC en diferentes pistas, tanto óvalos como procedimentales.

Primera Iteración

Al principio, las recompensas oscilaban entre **-100.0** y **100.0**. Esto resultó ser una mala práctica que sería corregida después. En *Deep Learning*, las recompensas deben oscilar entre intervalos pequeños (normalizados) para evitar que los pesos de las redes neuronales no se "saturen". Esto es lo que se conoce como *gradientes inestables*. Con valores extremos, un fallo catastrófico (como era salirse de la pantalla, **-100.0**) le "metía miedo" al algoritmo, disuadiéndolo de explorar, y una vuelta completa le hubiera dotado de una confianza extrema (**100.0**) sin asentar un progreso natural.

En esta iteración las recompensas variaban de la siguiente manera:

- **-100.0 puntos si el algoritmo conseguía escapar de la pantalla visible** (una visualización de 800 × 800 píxeles). Realmente esto era una salvaguarda contra cualquier cosa, porque el algoritmo el episodio se reseteaba al tocar la hierba.
- **-50.0 puntos por tocar la hierba (salirse del circuito)**. El objetivo era mantenerlo en la pista para que aprendiese a conducir, aunque fuese lento.
- **+10.0 por alcanzar un checkpoint**. Se intentaba promover el avance, ya que el coche se quedaba quieto de no hacerlo.
- **+100.0 puntos por completar el circuito**. El objetivo final merecía mucha recompensa.

Segunda Iteración

El coche aprendió a quedarse quieto tras alcanzar un par de checkpoints (e incluso a no moverse al inicio si "dudaba") para no perder su puntuación. Esto daba lugar a episodios infinitos, ya que no se había introducido todavía un límite de tiempo o de pasos. A las recompensas de antes se le añaden:

- **- 0.05 por paso**. Fomentaba el movimiento del coche, penalizando la actividad progresivamente. (Nótese la desproporción de escalas, otra mala práctica).

Tercera Iteración

El agente comenzó a moverse, pero a veces hacía giros bruscos que acababan en salidas de pista. Añadido a esto, el coche persistía en su estrategia de conseguir un par de recompensas y quedarse quieto. Añadido a los anteriores cambios:

- $-0.05 - (\text{grado de giro} * 0.1)$. Se buscaba minimizar los giros bruscos para que fuese más suave y previsor frente a una curva, pero permitiendo correcciones pequeñas.
- **Se implementa un sistema de micro-checkpoints.** $\text{distancia mejorada} * 0.5$ por cada unidad recorrida respecto al paso anterior.

Cuarta Iteración

A pesar de los cambios, el comportamiento de la inactividad continuaba. Sumado a la anterior:

- Se implementa un contador de inactividad con un límite de 300 pasos. -20.0 si el contador supera el límite de 300 pasos.

Con este cambio se consiguió que el coche, por fin, abandonase su estrategia y empezase a moverse, aunque acabara chocando.

Quinta Iteración

A la larga el algoritmo descubrió un vacío legal (Reward Hacking). Aprendió a resetear el contador de inactividad moviéndose un poco hacia adelante y luego dando marcha atrás. Sumado a los anteriores cambios:

- $(\text{velocidad} / 10.0) * 0.1$. Intentaba fomentar que el coche fuese más rápido, dándole una recompensa proporcional a su velocidad actual. Se complementó con la siguiente medida.
- -0.1 **si la velocidad es demasiado baja.** Esto pretendía atajar el problema de moverse muy poco a poco y de penalizar la marcha atrás.

Ninguno de los cambios citados hasta aquí dio un resultado demasiado prometedor, más allá del ya mencionado algoritmo de CNN en el óvalo.

Sexta Iteración (Cambio de CNN a MLP)

Como se ha explicado, al realizar el cambio de arquitectura (de imágenes a vectores), se aprovechó para cambiar el sistema de recompensas más limpio y escalado, intentando evitar la desestabilización de la red neuronal:

- Eliminada la marcha atrás. Para evitar de raíz el *Reward Hacking* de acelerar y dar marcha atrás para no ser penalizado por inactividad, se restringió el espacio de acciones. Ahora el coche solo podía avanzar, frenar y girar.
- -1.0 **por salirse de la pantalla.** De nuevo, una salvaguarda que a día de hoy no ha sido necesaria.
- -1.0 **por salirse de la pista.**
- $+1.0 + (\text{numero checkpoint actual} * 0.5)$. **Modificó el sistema de checkpoints.** Se premiaba el avance por el sistema de checkpoints haciendo la recompensa proporcional al número recorrido, contrarrestando así la penalización de dar pasos (si eran útiles) e incentivando al agente a llegar al final. De este modo el punto número 50 daba más puntos que el

número 30 y el algoritmo por fin entendería que llegar al final era el mejor premio de todos.

- **-5.0 por inactividad.** Seguía siendo el mayor enemigo y por ello fue necesario castigarlo más severamente.
- **-grados_giro * 0.01** . Se suavizó la penalización por girar bruscamente.
- Se eliminó el sistema de micro-checkpoints y su recompensa asociada.

Estas medidas, combinadas con la eficiencia de los vectores, dieron por fin los resultados que ya se han visto en el apartado anterior y conforman la distribución actual de la recompensa del proyecto.

3.3.2.- Curriculum Learning

El Curriculum Learning (Aprendizaje Curricular) es una técnica inspirada en el sistema educativo humano. Consiste en no enfrentar al agente al problema real desde el primer momento, sino en organizar un entorno de aprendizaje en el que se irá aumentando progresivamente la dificultad.

No se debe confundir con el Transfer Learning (Transferencia de Conocimiento), en la que se utiliza un modelo pre-entrenado para resolver una Tarea A para acelerar el entrenamiento para resolver una Tarea B (Ej: Usar un algoritmo que detecta gatos para entrenar un modelo que reconoce perros).

A nivel técnico, esta estrategia transfiere el conocimiento (los pesos de la red neuronal) de un modelo entrenado en un entorno hacia otro con las mismas reglas fundamentales, pero mayor complejidad (pasar del óvalo -simple, estático- a las pistas procedimentales - complejas, aleatorias-).

El Curriculum Learning no es estrictamente obligatorio, pero aporta beneficios a tener en cuenta:

- **Acelera la convergencia:** Al mostrarle al agente un entorno inicial sencillo, este solo tiene que preocuparse por descubrir las mecánicas básicas (acelerar, frenar, girar e interpretar las recompensas).
- **Reduce el riesgo de caer en óptimos locales:** Al bajar la dificultad, el agente no se abruma y no abandona a la primera de cambio. El ejemplo claro está en el punto anterior, dónde se ha mencionado la estrategia de recoger algún checkpoint (o incluso ninguno) y no moverse para evitar ser penalizado.

En el proyecto se ha implementado al entrenar los algoritmos en dos fases:

- **El óvalo:** Circuito estable, estático, simple. Su única función era que el algoritmo entendiese como se movía y como conseguía recompensas (y en el caso de usar la CNN enseñarle a ver).
- **El circuito procedimental:** Una vez el algoritmo demostraba saber navegar por el óvalo consistentemente se cargaba ese modelo pre-entrenado en los circuitos procedimentales. Así el agente no tenía que aprender desde cero, sino que podría dedicar

el 100% de su actividad computacional a aprender las diferentes curvas y generalizar la navegación.

4.- Conclusiones y futuro

La mayor conclusión técnica que hemos obtenido de este proyecto es que en el RL (como en la vida misma), la optimización y simplificación del entorno son la clave para el éxito. Cuantas menos variables existan, menos variables habrá que controlar. El uso de vectores y la simplificación de la arquitectura (de CNN a MLP) dio lugar a unos modelos capaces de generalizar y “conducir” de verdad.

A nivel personal, hemos experimentado en nuestras carnes que los algoritmos de RL requiere trabajo y paciencia, muchos ajustes y muchos “tirones de pelo” al ver que el agente ha encontrado otra manera de saltarse las reglas y seguir ganando recompensas sin cumplir ningún objetivo real. Es un aprendizaje sumamente frustrante y divertido a partes iguales, pero es lo que lo hace fascinante.

Llegado a este punto queda una pregunta en el aire: *¿Es imposible entrenar un modelo que funcione exclusivamente con imágenes?*

La respuesta rápida tras leer los problemas documentados en esta memoria podría parecer que es: Sí.

La respuesta real que se debería obtener al reflexionar sobre la naturaleza del Deep Learning y lo expuesto en este documento es: *¡Para nada, sólo necesitábamos más tiempo y capacidad de cómputo!*

Entrenar con imágenes fue la idea principal, tanto por la fascinación que suponía “enseñar a una máquina a ver”, como porque era lo que decía originalmente el enunciado. Pero lo cierto es que para ser un primer trabajo de RL era una tarea titánica partiendo de la base conocida (que era nula).

Aún queda la espinita clavada de no haberlo conseguido y seguramente sigamos explorándolo cuando tengamos más tiempo y/o mejores recursos. El paper de Simeet Nayan mencionado en el punto 3.1.-Usando Imágenes y CNN, es sumamente interesante y aporta claves bastante importantes para la consecución de este objetivo.

Así que si hay un cambio que pide a gritos este proyecto es exactamente ese.